



LinkedIn Clone: LinkedIn-like платформа

Frontend-частина дипломного проекту професійного нетворкінгу

- Формат звіту: кожен слайд пояснює причину вибору рішення та результат
- Проект: full-stack LinkedIn-like платформа з інтеграцією frontend ↔ backend
- Frontend розробники: Ямчук Тимур, Андрій Ротарь
- стек: React 19, Vite 6, ASP.NET Core, PostgreSQL, JWT, SignalR
- Демо на захисті: andrii.rotar@gmail.com, admin@local.dev, `verify:e2e 17/17`

1. Звіт: Вступ: чому саме LinkedIn

Обґрунтування рішення: LinkedIn обрано як найсильнішу модель, бо він поєднує професійний профіль, мережу контактів, контент, вакансії та комунікацію.

6

основних розділів об'єднані в
одному сайті

- Причина вибору: Це не просто сайт з вакансіями, а повна професійна екосистема для кандидата, роботодавця і спільнот.
- Обране рішення: Такий формат дозволяє показати більше дипломної роботи: UI, маршрути, клієнтські стани та користувацькі сценарії.
- Реалізація: На захисті показуємо live-flow через реальний API: вхід → профіль → стрічка → мережа → вакансії → чати.

2. Звіт: Файлова структура проєкту

Звітний огляд: Фокус доповіді — frontend SPA; backend показано лише для розуміння масштабу системи

frontend/ — наш фокус

```
frontend/  
├── src/  
│   ├── app/ router, layout, theme, i18n, transitions  
│   ├── pages/ auth, home, network, vacancies, messages, admin  
│   ├── features/ auth, profile, jobs, messaging, network, content  
│   └── shared/  
│       ├── api/ HTTP-клієнт, tokens, refresh, paths  
│       ├── ui/ спільні компоненти  
│       └── lib/ helpers, pagination, session  
├── scripts/verify-e2e.mjs  
└── package.json
```

backend/ — контекст

```
backend/ (контекст масштабу)  
├── Identity · Profile · Professional  
├── Network · Content · Messaging  
├── Jobs · Events · Notifications  
└── Facade.API
```

Структура з реального репозиторію: React SPA розбито на pages, features, app та shared.

3. Звіт: Порівняння з конкретними конкурентами

Стан у конкурентів: Що є у Work.ua, LinkedIn, Djinni, Robota.ua

- Аналіз: Work.ua: сильний пошук вакансій, фільтри, резюме та відгуки, але фокус переважно на job-board сценарії.
- Аналіз: LinkedIn: сильна глобальна мережа, профіль, контент і вакансії, але продукт перевантажений для локального навчального MVP.
- Аналіз: Djinni: зручний IT-рекрутинг і анонімний профіль, але вузький фокус на наймі без широких спільнот і стрічки.
- Аналіз: Robota.ua: багато вакансій, резюме і роботодавців, але менше соціальної взаємодії, чатів і професійного контенту.

Наше рішення: Чого бракує там, але є у LinkedIn Clone

- Перевага: У LinkedIn Clone вакансії не ізольовані: вони пов'язані з профілем, чатами, контактами і загальним шляхом кандидата.
- Перевага: LinkedIn Clone бере логіку LinkedIn, але показує її у компактному MVP, де кожен модуль зрозумілий на захисті.
- Перевага: У LinkedIn Clone є не тільки найм, а й контент, groups/pages/events, глобальний пошук і особистий кабінет.
- Перевага: Наш сайт об'єднує job-board, професійну соцмережу і внутрішню комунікацію в одному frontend-сценарії.

4. Звіт: Переваги UX нашого сайту

Звітний висновок: Реалізовано в поточній версії сайту на гілці merger



Єдиний UX

Рішення: одна навігація для всіх сценаріїв



Dark/Light

Рішення: перемикання теми через CSS variables



i18n

Рішення: 4 мови інтерфейсу в runtime



Картки

Рішення: зрозуміла структура контенту



Анімації

Рішення: AnimatedOutlet — плавні переходи сторінок



Пошук

Рішення: HeaderSearch по профілях і контенту



Live API

Рішення: JWT, REST, refresh, SignalR messaging



E2E verify

Рішення: 17 автоматичних smoke-тестів інтеграції



Єдиний UX: темна/світла тема, 4 мови, плавні переходи та live backend API.

5. Звіт: Що є у конкурентів і що додано у нас

Стан у конкурентів: У конкурентів

- Аналіз: Вакансії існують окремо від соціальної активності кандидата.
- Аналіз: Пошук роботи часто не пов'язаний з чатами та мережею контактів.
- Аналіз: Демонстраційні проекти нерідко залишаються статичними макетами.
- Аналіз: Інтерфейс не завжди показує повний шлях користувача від входу до дії.

Наше рішення: У нашому LinkedIn Clone

- Перевага: Користувач шукає роботу, веде профіль, читає контент і спілкується в одному місці.
- Перевага: Вакансії, контакти та повідомлення об'єднані в логічний кандидатський сценарій.
- Перевага: Frontend підключений до Facade.API: auth, profile, feed, network, jobs, messaging.
- Перевага: На захисті показуємо live-flow: login → profile → feed → network → vacancies → messages.

6. Звіт: Компонентна структура frontend

Звітний огляд: Кожен блок — ізольований React-модуль з власною логікою

Дерево папок frontend

```
frontend/
├── src/
│   ├── app/ router, layout, theme, i18n, transitions
│   ├── pages/ auth, home, network, vacancies, messages, admin
│   ├── features/ auth, profile, jobs, messaging, network, content
│   ├── shared/
│   ├── api/ HTTP-клієнт, tokens, refresh, paths
│   ├── ui/ спільні компоненти
│   ├── lib/ helpers, pagination, session
│   ├── scripts/verify-e2e.mjs
│   └── package.json
```

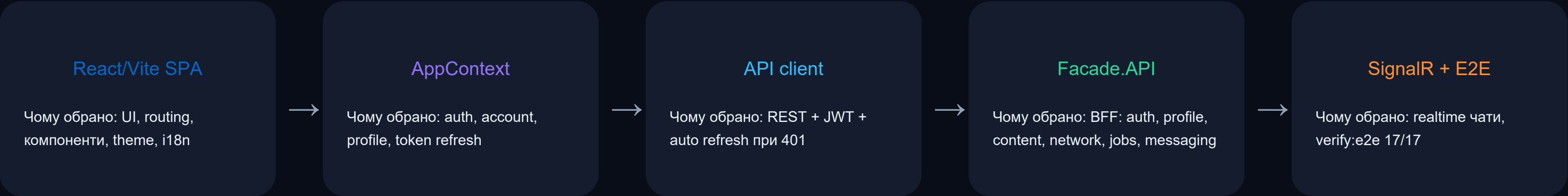
Ізольовані React-модулі

- ✓ Модуль: pages/ — auth, home, network, vacancies, messages, notifications, admin
- ✓ Модуль: features/ — API-виклики, mappers, enrich* для профілів і постів
- ✓ Модуль: app/ — router, AppContext, Layout, theme, i18n, page transitions
- ✓ Модуль: shared/api/ — http, tokens, refresh при 401, централізовані paths
- ✓ Модуль: AppContext — auth, account, profile в одному провайдері замість legacy stores

Архітектура: Код розбито за папками так, щоб модулі не залежали один від одного напряму.

7. Звіт: Архітектура frontend

Frontend: Context API, API-клієнт, демо-режим



Результат рішення: Frontend і backend працюють як одна система: SPA викликає REST API, SignalR для messaging, PostgreSQL на сервері.

8. Звіт: Маршрутизація frontend

SPA з guarded routes, state providers, темою та i18n

/auth

Роль у рішенні: login, register, Google/Facebook OAuth

ProtectedRoute

Роль у рішенні: захист /app/* маршрутів

AppContext

Роль у рішенні: account, profile, token refresh при завантаженні

/app/*

Роль у рішенні: home, network, vacancies, messages, profile

AnimatedOutlet

Роль у рішенні: плавні fade-переходи між сторінками

redirect / → /app

Роль у рішенні: стартовий сценарій після входу

9. Звіт: Live-flow для захисту

Демонстраційний шлях користувача на захисті



Login (JWT)

Профіль

Стрічка

Мережа

Вакансії

Чати (SignalR)

Звітний результат: На захисті показуємо live-сценарій: backend localhost:5000, frontend localhost:5173, demo-акаунт andrii.rotar@gmail.com.

10. Звіт: Адмін-панель платформи

Звітний огляд: Окремий UI-контур /admin для модерації та керування LinkedIn Clone

Доступ і архітектура

- ✓ /admin/* → RequireAuth + RequireAdmin
- ✓ JWT role Admin (user.isAdmin)
- ✓ Без ролі — AdminForbiddenPage (403)
- ✓ AdminLayout: sidebar з 7 розділами
- ✓ adminApi.js → /api/admin/*

Розділи адмін-панелі

DB	Dashboard	stats: users, posts, vacancies, events	✓
US	Users	list, filters, lock/unlock, roles, create user	✓
CT	Content	moderation постів: search, delete/restore	✓
CM	Comments	moderation коментарів до постів	✓
JB	Jobs	moderation вакансій + recommended queries	✓
EV	Events	moderation подій: delete/restore	✓
RL	Roles	ролі User/Admin у drawer користувача	✓

Результат: Admin-панель працює з реальним backend: admin@local.dev, JWT role Admin, /api/admin/* endpoints.

11. Звіт: Розподіл frontend-роботи

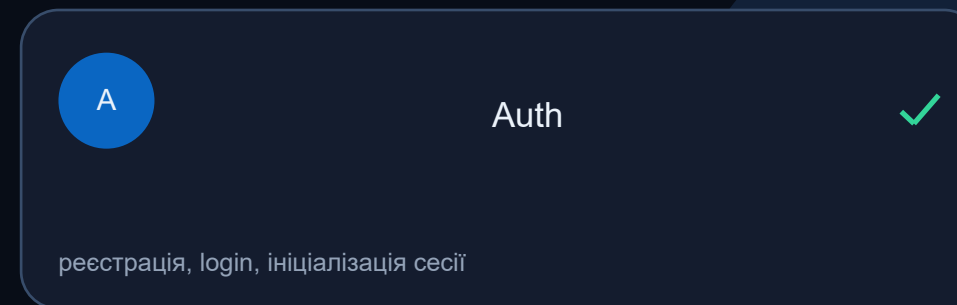
Ямчук Тимур

- Зона відповідальності: Вікно реєстрації та auth UX.
- Зона відповідальності: Сторінка вакансій і пошук роботи.
- Зона відповідальності: Чати, дизайн і особистий кабінет.

Андрій Ротарь

- Зона відповідальності: Home/Feed: AppContext, оптимізація рендерингу стрічки, динамічні реакції.
- Зона відповідальності: Network: контакти, groups, pages, events, EventPanel.
- Зона відповідальності: API-клієнт, backend-інтеграція та E2E verify (17/17).

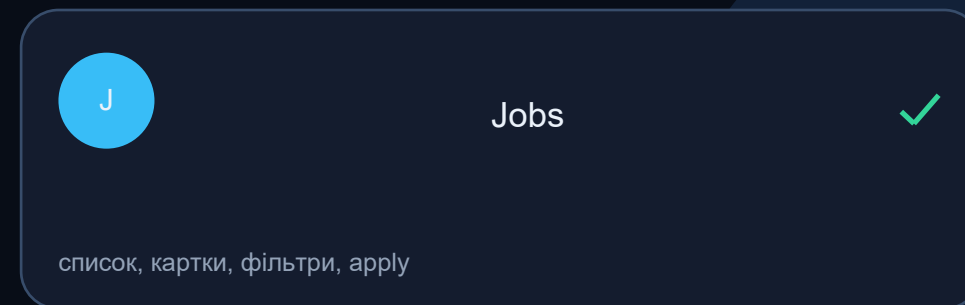
12. Звіт: Ямчук Тимур: вікно реєстрації



Обґрунтування рішення: Реєстрація є першою точкою входу користувача в платформу.

- Причина вибору: Login/register через `/api/auth/\*` з JWT access + refresh tokens.
- Обране рішення: Google і Facebook OAuth кнопки; demo: andrii.rotar@gmail.com / LinkUpDemo2024!
- Реалізація: AppContext зберігає account і profile після входу; auto refresh токена.
- Результат: Auth-екран узгоджений з темною/світлою темою та і18п.

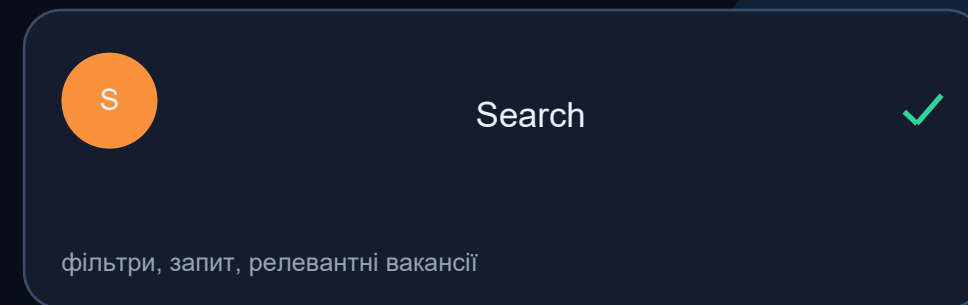
13. Звіт: Ямчук Тимур: сторінка вакансій



Обґрунтування рішення: Сторінка вакансій реалізує один з головних сценаріїв LinkedIn-like платформи.

- Причина вибору: Список вакансій завантажується з `/api/jobs/vacancies` — реальні дані з backend.
- Обране рішення: AI-рекомендації в sidebar: підбір за навичками профілю (Java, PostgreSQL) + VacancyDetailModal.
- Реалізація: Фільтри, збереження вакансій і apply flow підключені до jobs API.
- Результат: Кнопки «Детальніше» та «Бути серед кандидатів» — повний сценарій відгуку.

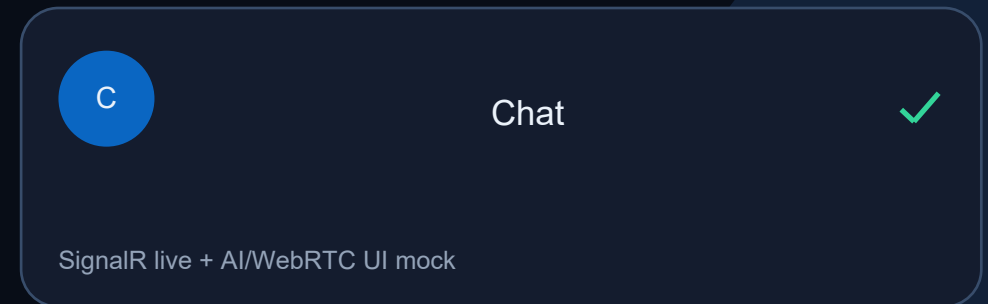
14. Звіт: Ямчук Тимур: пошук роботи



Обґрунтування рішення: Пошук роботи робить модуль вакансій корисним для щоденного сценарію кандидата.

- Причина вибору: Реалізовано фільтрацію вакансій за ключовими параметрами.
- Обране рішення: Опрацьовано сценарій, коли результатів немає або користувач змінює запит.
- Реалізація: Пошук допомагає швидко перейти від загального списку до релевантних вакансій.
- Результат: Цей блок підсилює головну перевагу сайту: вакансії пов'язані з профілем і комунікацією.

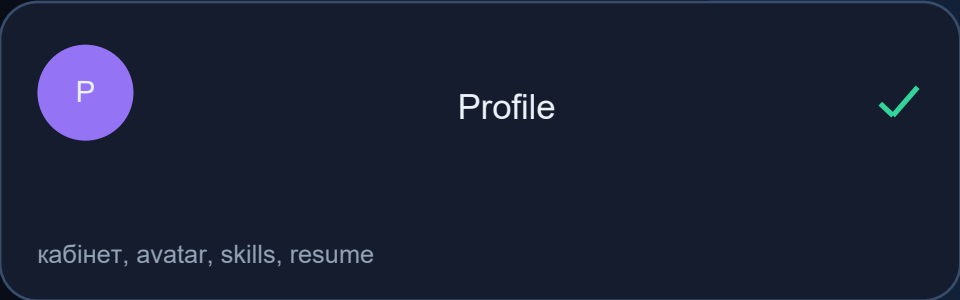
15. Звіт: Ямчук Тимур: чати



Обґрунтування рішення: Чати — live messaging через SignalR; AI-асистент і дзвінки — UI-шар для наступних релізів.

- Причина вибору: MessageFiltersModal: фільтри Unread, Contacts, Favorites, Spam, Archive, Drafts.
- Обране рішення: SignalR hub для realtime повідомлень; unread, archive, mute, delete, share posts — live API.
- Реалізація: AI assistant і VoiceCallOverlay: повністю спроектований UI-шар; на захисті — інтерактивні mock-моделі.
- Результат: Live WebRTC інтеграція запланована на наступні релізи; core messaging уже працює з backend.

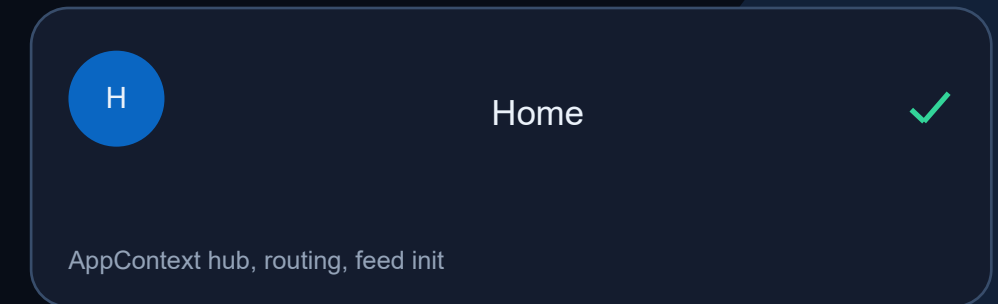
16. Звіт: Ямчук Тимур: дизайн і особистий кабінет



Обґрунтування рішення: Особистий кабінет і дизайн формують завершене враження від frontend-частини.

- Причина вибору: Профіль синхронізується з ``/api/profile/me``: `headline`, `bio`, `avatar upload/delete`.
- Обране рішення: `Skills`, `experience`, `education` — CRUD через professional API; demo skills seeder.
- Реалізація: `ProfileCompletion` ring, `resume upload`, public profile ``/app/profile/:userId``.
- Результат: `Dark/light theme`, `i18n (uk/en/de/es)` — єдиний візуальний стиль платформи.

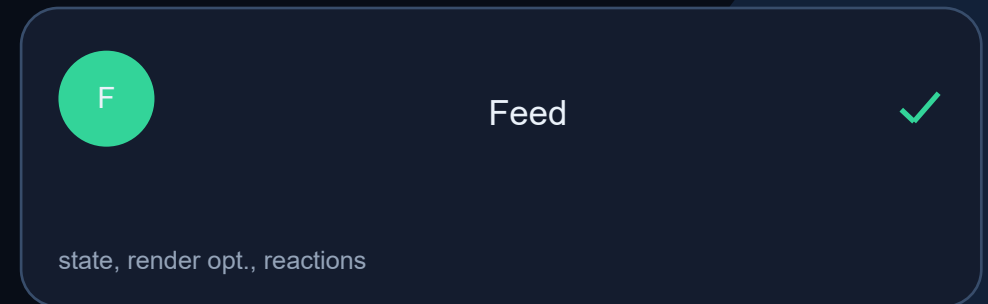
17. Звіт: Андрій Ротарь: головний екран



Обґрунтування рішення: Home — архітектурний hub після auth: зв'язує AppContext, routing і завантаження Feed.

- Причина вибору: Стартова точка після login: orchestration layout, sidebar, швидкий доступ до модулів.
- Обране рішення: AppContext передає account/profile у Home без prop drilling через legacy stores.
- Реалізація: Home ініціює завантаження стрічки з content API — перехід до найскладнішого модуля Feed.
- Результат: Єдиний UX-hub: мережа, вакансії, повідомлення, профіль з одного екрана.

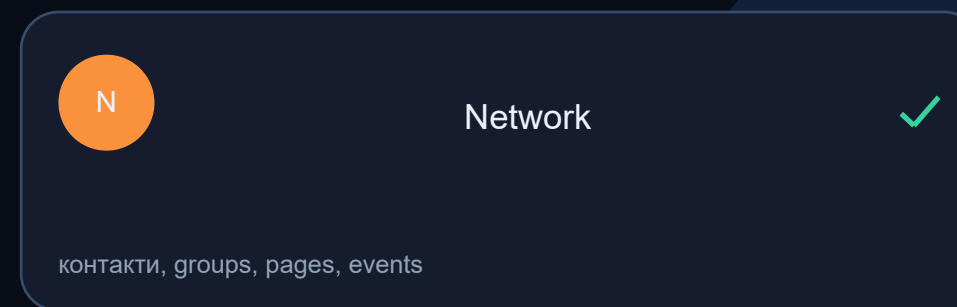
18. Звіт: Андрій Ротарь: контентна стрічка (Feed)



Обґрунтування рішення: Feed — найскладніший архітектурний модуль: глобальний стан, рендеринг і динамічні реакції.

- Причина вибору: AppContext + локальний state: пости з content API, enrichPostsWithAuthors без full reload.
- Обране рішення: Оптимізація рендерингу: PostCard, reactions, comments оновлюються точково після дії користувача.
- Реалізація: PostEngagement, PostCommentsPanel, PostShareMenu — повний цикл взаємодії з backend.
- Результат: CreatePost modal, rich text, media — технічно складніший блок, ніж статичні auth/profile форми.

19. Звіт: Андрій Ротарь: архітектура Network



Обґрунтування рішення: Network: контакти, запити, suggestions і вкладка «Активність» з events.

- Причина вибору: Вкладки «Нові контакти» та «Активність»: incoming/outgoing requests, people you may know.
- Обране рішення: EventPanel: фільтри All/Upcoming/Online/In-person, discover events з API.
- Реалізація: AddContactSearch, ManageNetworkModal, networkPageCache для плавного UX.
- Результат: Groups, pages, company/event routes — ``/app/groups/:id``, ``/app/event/:id``.

20. Звіт: Андрій Ротарь: API-клієнт і backend-інтеграція



Арі-Клієнт



арі-клієнт і backend-інтеграція

Обґрунтування рішення: Frontend підключений до Facade.API — auth, profile, content, network, jobs, messaging.

- Причина вибору: http.js: Bearer token, auto refresh при 401, централізовані paths і error handling.
- Обране рішення: verify-e2e.mjs: 17 автоматичних smoke-тестів (auth, profile, feed, network, jobs, chats).
- Реалізація: SignalR messaging hub; enrich* helpers для профілів у постах, чатах, notifications.
- Результат: Docker Compose: Postgres + API :5000 + Vite :5173 — відтворюваний демо-стек.

21. Звіт: Висновок по frontend-роботі

Причина / виконання: Що реалізовано

- Звіт: Full-stack MVP: UI + REST API + JWT + SignalR + PostgreSQL.
- Звіт: Два зони відповідальності: кандидатський UX (Тимур) і контент/мережа/API-клієнт (Андрій).
- Звіт: E2E verify 17/17, admin panel, AI job recommendations, message filters.

Результат / висновок: Практичний результат

- Висновок: Це працездатний LinkedIn-like продукт, а не статичний макет.
- Висновок: Frontend і backend інтегровані на гілці merger — live demo на захисті.
- Висновок: Roadmap: events seeding, OAuth production, integration tests.



22. Звіт: Підсумок

Full-stack MVP з live API — готовий продукт для захисту

Підсумковий висновок: LinkedIn Clone — full-stack професійна екосистема: frontend + backend + PostgreSQL.

Підсумковий висновок: На захисті демонструємо live-flow: login → profile → feed → network → jobs → chat.

Підсумковий висновок: Головний результат — інтегрований продукт з E2E verify 17/17 і зрозумілим UX.

